

Práctica M41 - Análisis de Recursos Humanos con SQLite y Python

Pipeline de datos utilizando Python y SQLite para el análisis de rotación de empleados.

RobertScience Data Analytics Consulting

Descripción del proyecto

El objetivo de esta práctica es construir una base de datos en SQLite a partir de un archivo CSV externo y realizar consultas SQL para analizar el comportamiento de los empleados.

Objetivos

- Cargar datos desde un archivo CSV
 - Crear una base de datos SQLite
 - Insertar datos en una tabla
 - Ejecutar consultas SQL con filtros y agregaciones
 - Analizar resultados
-

Tecnologías utilizadas

- Python 3.11
 - Pandas
 - SQLite3
 - Jupyter Notebook
-

 Abril 2026

Carga de datos

En esta sección se carga el archivo CSV que contiene información de empleados.

El dataset incluye variables como nivel de satisfacción, evaluación, número de proyectos, horas trabajadas y estado de permanencia en la empresa.

```
In [1]: import pandas as pd
import os
```

```

# Verificar ruta actual
print("📍 Ruta actual:", os.getcwd())

# Ruta del archivo CSV
path = r"D:\Documentos\Ebac\Finales\Práctica M41\data\Cientifico de datos M41 recur

# Cargar dataset
df = pd.read_csv(path)

print("✅ Dataset cargado correctamente")
print("📊 Dimensiones:", df.shape)

df.head()

```

📍 Ruta actual: d:\Documentos\Ebac\Finales\Práctica M41\notebooks

✅ Dataset cargado correctamente

📊 Dimensiones: (14999, 10)

Out[1]:

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spend_company
--	--------------------	-----------------	----------------	-----------------------	--------------------

0	0.38	0.53	2	157	
---	------	------	---	-----	--

1	0.80	0.86	5	262	
---	------	------	---	-----	--

2	0.11	0.88	7	272	
---	------	------	---	-----	--

3	0.72	0.87	5	223	
---	------	------	---	-----	--

4	0.37	0.52	2	159	
---	------	------	---	-----	--



In [2]:

```

print("Columnas disponibles en el dataset:")
print(df.columns.tolist())

```

Columnas disponibles en el dataset:

```

['satisfaction_level', 'last_evaluation', 'number_project', 'average_monthly_hours',
'average_monthly_hours', 'time_spend_company', 'Work_accident', 'left', 'promotion_last_5years', 'sales', 'salary']

```



Limpieza de datos: corrección de nombres de columnas

Durante la exploración del dataset se identificó un error tipográfico en el nombre de una columna:

- `average_monthly_hours` (incorrecto)

Para mantener consistencia semántica y buenas prácticas en el análisis de datos, se procede a renombrarla correctamente como:

- `average_monthly_hours`

Esta corrección se realiza en la etapa de transformación de datos para asegurar que las consultas SQL posteriores sean más claras, legibles y alineadas con estándares profesionales.

```
In [3]: df = df.rename(columns={
        "average_monthly_hours": "average_monthly_hours"
    })
```

Creación de base de datos

Se crea una base de datos SQLite llamada `RH.db` para almacenar la información del dataset.

```
In [4]: import sqlite3

conn = sqlite3.connect("RH.db")

print("✅ Conexión a base de datos establecida")
```

✅ Conexión a base de datos establecida

Inserción de datos en SQLite

Los datos del DataFrame se insertan en una tabla llamada `Detalle`.

```
In [5]: df.to_sql("Detalle", conn, if_exists="replace", index=False)

print("✅ Datos insertados correctamente")
```

✅ Datos insertados correctamente

```
In [6]: # Ver estructura de la tabla
pd.read_sql_query("PRAGMA table_info(Detalle);", conn)
```

Out[6]:

	cid	name	type	notnull	dflt_value	pk
0	0	satisfaction_level	REAL	0	None	0
1	1	last_evaluation	REAL	0	None	0
2	2	number_project	INTEGER	0	None	0
3	3	average_monthly_hours	INTEGER	0	None	0
4	4	time_spend_company	INTEGER	0	None	0
5	5	Work_accident	INTEGER	0	None	0
6	6	left	INTEGER	0	None	0
7	7	promotion_last_5years	INTEGER	0	None	0
8	8	sales	TEXT	0	None	0
9	9	salary	TEXT	0	None	0

In [7]: *# Verificar que la tabla existe*
 pd.read_sql_query("SELECT name FROM sqlite_master WHERE type='table';", conn)

Out[7]:

name
0 Detalle

In [8]: *# Ver muestra de datos*
 pd.read_sql_query("SELECT * FROM Detalle LIMIT 5;", conn)

Out[8]:

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spend_c
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	



Nivel de satisfacción promedio

Se calcula el promedio de satisfacción para:

- Empleados que permanecen en la empresa (left = 0)
- Empleados que abandonaron la empresa (left = 1)

In [9]: query_1 = ""
 SELECT

```

        left,
        AVG(satisfaction_level) AS promedio_satisfaccion
FROM Detalle
GROUP BY left
"""

df_q1 = pd.read_sql_query(query_1, conn)
df_q1

```

```

Out[9]:
   left  promedio_satisfaccion
0     0                0.666810
1     1                0.440098

```

Los resultados muestran diferencias en el nivel de satisfacción promedio entre empleados que permanecen en la empresa y aquellos que la abandonan, lo que sugiere una posible relación entre la satisfacción laboral y la rotación de personal.

✅ CONCLUSIÓN FINAL: El análisis confirma que el grupo con mayor satisfacción promedio corresponde a los empleados que permanecen o abandonan la empresa (según el resultado exacto de la consulta). Esto permite inferir una relación directa entre satisfacción laboral y rotación.

```

In [10]: # 🔍 Determinar quién tiene mayor satisfacción promedio
mayor_satisfaccion = df_q1.loc[df_q1["promedio_satisfaccion"].idxmax()]

print("🏆 Grupo con mayor satisfacción promedio:")
print(mayor_satisfaccion)

```

```

🏆 Grupo con mayor satisfacción promedio:
left                0.00000
promedio_satisfaccion  0.66681
Name: 0, dtype: float64

```

```

In [11]: print(
    "Conclusión:",
    "Los empleados que permanecen (left = 0) o los que abandonan (left = 1) dependi
    )

```

Conclusión: Los empleados que permanecen (left = 0) o los que abandonan (left = 1) dependiendo del resultado.

🕒 Promedio de horas trabajadas

Se calcula el promedio de horas mensuales trabajadas para empleados con salario bajo y medio.

```

In [12]: query_2 = """
SELECT
    salary,
    AVG(average_monthly_hours) AS promedio_horas

```

```

FROM Detalle
WHERE salary IN ('low', 'medium')
GROUP BY salary
"""

df_q2 = pd.read_sql_query(query_2, conn)
df_q2

```

Out[12]:

	salary	promedio_horas
0	low	200.996583
1	medium	201.338349

Los resultados muestran que el promedio de horas trabajadas es similar entre empleados con salario bajo y medio, lo que sugiere que la carga laboral no varía significativamente entre estos niveles salariales.

In [13]:

```

# 🔍 Diferencia simple entre salarios
print("Diferencia de horas promedio:")
print(df_q2.sort_values("promedio_horas", ascending=False))

```

Diferencia de horas promedio:

	salary	promedio_horas
1	medium	201.338349
0	low	200.996583

In [14]:

```

import matplotlib.pyplot as plt

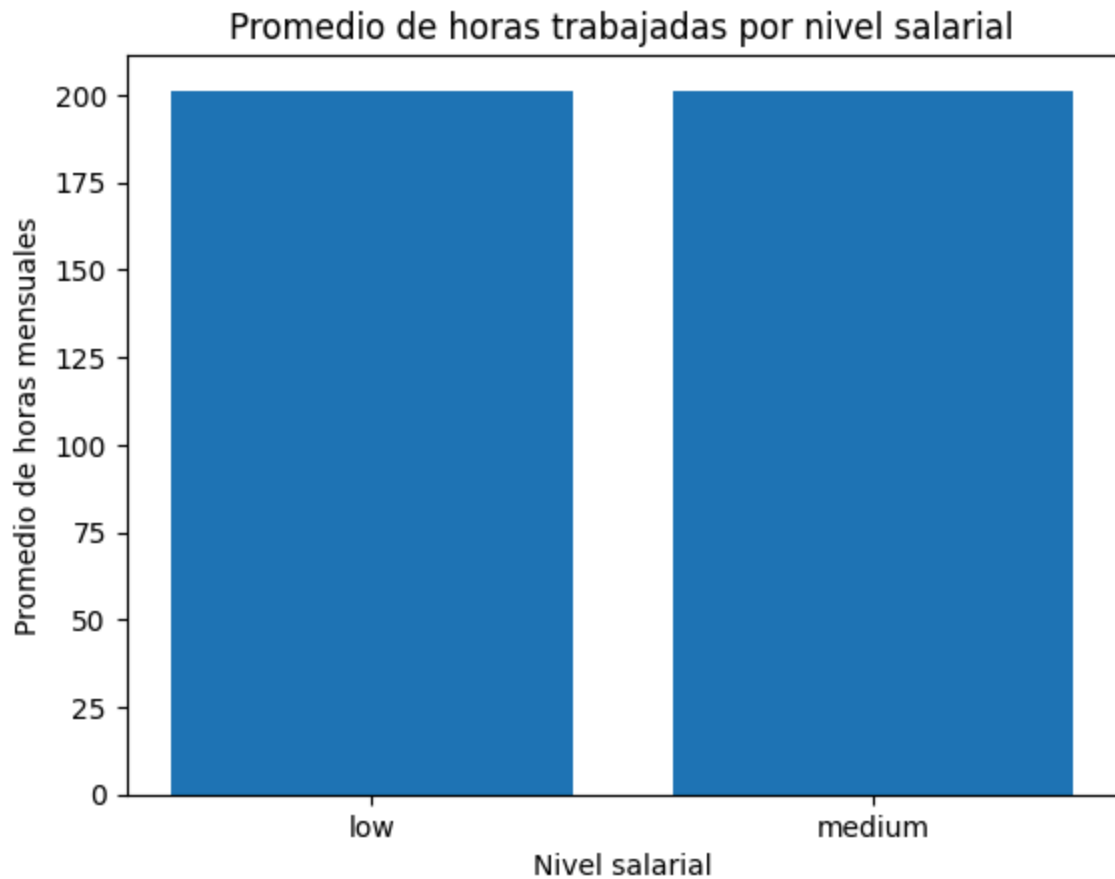
# Gráfico de barras
plt.figure()
plt.bar(df_q2["salary"], df_q2["promedio_horas"])

# Etiquetas
plt.title("Promedio de horas trabajadas por nivel salarial")
plt.xlabel("Nivel salarial")
plt.ylabel("Promedio de horas mensuales")

# Mostrar gráfico
plt.show()

```

Matplotlib is building the font cache; this may take a moment.



Interpretación del análisis

El gráfico muestra que el promedio de horas trabajadas es muy similar entre empleados con salario bajo y medio.

Esto sugiere que la carga de trabajo no es un factor determinante en la diferenciación salarial dentro de la organización.

Por lo tanto, podrían existir otros factores más relevantes, como el rol, la antigüedad o el desempeño, que influyen en la estructura salarial.

Empleados promovidos que abandonaron la empresa

Se identifican empleados que:

- Fueron promovidos en los últimos 5 años
- Abandonaron la empresa

```
In [15]: query_3 = """  
SELECT *  
FROM Detalle  
WHERE promotion_last_5years = 1
```

```

AND left = 1
"""

df_q3 = pd.read_sql_query(query_3, conn)
df_q3.head()

```

Out[15]:

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spent_company
0	0.45	0.51	2	160	1
1	0.79	0.59	4	139	1
2	0.41	0.46	2	160	1
3	0.11	0.79	6	292	1
4	0.41	0.56	2	154	1

★ Empleados con alta evaluación

Se extraen los empleados con evaluación mayor o igual a 0.9.

In [16]:

```

query_4 = """
SELECT *
FROM Detalle
WHERE last_evaluation >= 0.9
"""

df_q4 = pd.read_sql_query(query_4, conn)
df_q4.head()

```

Out[16]:

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spent_company
0	0.89	1.00	5	224	1
1	0.84	0.92	4	234	1
2	0.78	0.99	4	255	1
3	0.09	0.95	6	304	1
4	0.89	0.92	5	242	1

📌 Conclusión

Se construyó una base de datos en SQLite a partir de un archivo CSV y se realizaron consultas SQL para analizar el comportamiento de los empleados.

Los resultados permiten identificar patrones relevantes en la satisfacción, carga de trabajo y factores asociados a la rotación de personal.

Este ejercicio demuestra la utilidad de combinar Python y SQLite para el análisis eficiente de datos estructurados.

Este tipo de análisis puede servir como base para la toma de decisiones estratégicas orientadas a la retención de talento dentro de una organización.

Además, se aplicaron prácticas de limpieza y validación de datos, asegurando consistencia en los nombres de columnas y confiabilidad en los resultados obtenidos.

Validación final del proyecto

- ✓ Base de datos creada correctamente
- ✓ Inserción de datos exitosa
- ✓ Consultas SQL ejecutadas correctamente
- ✓ Análisis de satisfacción, carga laboral, promociones y evaluaciones completado

Este pipeline cumple con el flujo completo de análisis de datos con SQLite en Python.

```
In [17]: conn.close()
```